

Optional Introductory Material

This note discusses the basics of Pandas. If you have used Pandas for data analysis, you can skip this part.

Pandas

Pandas provides high-performance, easy-to-use data structures and data analysis tools. Because of its expressive syntax and efficient handling of large datasets—backed by highly optimized C and NumPy, pandas has become the de facto standard for data wrangling and analysis in Python.

At its core are two primary building blocks:

- Series: A one-dimensional, homogeneously typed array with labeled indices.
- DataFrame: A two-dimensional, tabular data structure with labeled axes (rows and columns), allowing heterogeneous types across columns. -

```
import pandas as pd
```

Pandas Series

- A pandas Series is a one-dimensional labelled array
- A series combines the best features of a list and a dictionary
- A series maintains a single collection of ordered values
- We can assign each value an identifier, which does not have to be unique

Create a Series

```
lst = [1,2,3,4,5]
pd.Series(lst)      # from a List
```

```
0    1
1    2
2    3
3    4
4    5
dtype: int64
```

```
dict_raw = {
    "Nikhil": 13,
    "Rohan": 14,
    "Sharmila": 45,
    "Vishal": 48
}
dict_series = pd.Series(dict_raw)      # from a dictionary
dict_series
```

```
Nikhil      13
Rohan       14
Sharmila    45
Vishal     48
dtype: int64
```

Methods

- Syntax for Series methods is series_object.method()
- For example: series_object.sum()

```
coffee_expense = pd.Series([4,6,4,4,7,0,0])
coffee_expense.mean()      # Mean of Series
coffee_expense.std()       # Standard deviation of Series
coffee_expense.sum()       # Sum of Series
```

```
np.int64(25)
```

Attributes

- An attribute is a piece of data that lives on an object
- An attribute is a fact, a characteristic of the object
- Access an attribute with `object.attribute`

In contrast to an Attribute, a Method is a behavior or action of an object. A Method is like a command, while an Attribute is a characteristic. Asking a student to complete a Python code exercise is a method while their name is an attribute.

Note the numeric identifier for each element

```
coffee_expense.dtype    # Data type
coffee_expense.size     # Size
coffee_expense.is_unique # Are all values unique
coffee_expense.values   # Values
coffee_expense.index    # Index
```

DataFrames

- Two-dimensional data structure that resembles a typical spreadsheet
- Pandas uses NaN to designate missing values. It is short for Not a Number
- Similar to Series, Pandas assigns an index position/label to each DataFrame row
- DataFrames share a number of methods and attributes with Series. However, there are a few unique ones such as **hasnans** attribute which exists only for Series, and **columns** attribute which only works with a DataFrame
- Some methods/attributes will return a different data type
- **info** method returns a summary of a Pandas object

It is worth noting that simply using square brackets for indices (as shown below) works but the recommended option is to specify `loc` method for an index or `iloc` for an index location.

```
dict_raw = {'names': ['Nikhil', 'Rohan', 'Sharmila', 'Vishal'],
            'gender': ['Male', 'Male', 'Female', 'Male'],
            'favorite_fruit': [None, 'Pomegranate', 'Mango', 'Pomegranate'],
            'sleep_hours': [8, 7, 6, 6]}
data = pd.DataFrame(dict_raw)
data
```

	names	gender	favorite_fruit	sleep_hours
0	Nikhil	Male	None	8
1	Rohan	Male	Pomegranate	7
2	Sharmila	Female	Mango	6
3	Vishal	Male	Pomegranate	6

Key Methods

- `.head()`
- `.tail()`
- `.info()`: Structure of the data
- `.describe()`

```
data.info()
```

Key Attributes

- `.index`
- `.values`: will display values as an array of arrays
- `.shape`
- `.dtypes`: Will list type of each column/variable. Pandas refers to string as object. If the data contains missing values (NaN), the column is treated as a floating point data type (as opposed to say integer).
- `.columns`: List of all column indices

- `.axes`: List of all indices. This would include row indices and column indices for a two-dimensional structure.

```
data.shape  
data.columns
```

To illustrate, when one uses `.loc` accessor for a non-existent column name like calling “US” instead of “United States”, it results in an error. On the other, hand, `get` Method can generate a custom error message.

Summary

This note introduced Pandas Series and DataFrame